

90 DAYS CHALLENGE • DAY 06

Python Notes — Control Flow (Nested Conditions, Ternary, match-case, Truthy & Falsy)

7.4 Nested Conditions

Definition: A nested condition means writing one condition inside another condition. The inner condition is checked only when the outer condition is **True**. It is useful when one decision depends on another decision.

Syntax

```
if outer_condition:
    if inner_condition:
        # statements
    else:
        # statements
else:
    # statements
```

Example

```
age = 20
has_ticket = True

if age >= 18:
    if has_ticket:
        print("Entry allowed")
    else:
        print("Ticket required")
else:
    print("Entry denied")
```

Output: Entry allowed

Flow: First, Python checks whether the person is at least 18. If True, it checks whether the person has a ticket.

7.5 Ternary Operator

The ternary operator is a short way to write a simple **if-else** statement in one line. It is useful when choosing between two values. Use it for simple conditions; for complex logic, normal if-else is clearer.

Syntax

```
value_if_true if condition else value_if_false
```

Normal if-else

```
age = 20
if age >= 18:
    status = "Adult"
else:
    status = "Minor"
print(status)
```

Output: Adult

Same code using the ternary operator

```
age = 20
status = "Adult" if age >= 18 else "Minor"
print(status)
```

Output: Adult

7.6 match-case

Definition: match-case compares one value with multiple possible patterns or cases. The case `_` acts like a default case and runs when no earlier case matches. It was introduced in Python 3.10.

Syntax

```
match value:
    case pattern1:
        # statements
    case pattern2:
        # statements
    case _:
        # default statements
```

Example

```
day = 3
match day:
    case 1:
        print("Monday")
    case 2:
        print("Tuesday")
    case 3:
        print("Wednesday")
    case _:
        print("Invalid day")
```

Output: Wednesday

7.7 Truthy and Falsy Values

Python conditions do not always need a direct comparison such as `if age >= 18:`. Python can evaluate a value as True or False in a condition. Values that behave like True are called **truthy**; values that behave like False are called **falsy**.

Common falsy values

Value	Meaning
False	Boolean false
0	Zero integer
0.0	Zero float
""	Empty string
None	No value / absence of a value

Common truthy values

Value	Meaning
True	Boolean true
10, -5	Non-zero numbers
"Python"	Non-empty string
" "	String containing a space

Important: `" "` is truthy because it contains a space. It is not an empty string.

Example 1: Non-empty string

```
name = "Rahul"
if name:
```

```
    print("Name is available")
else:
    print("Name is missing")
```

Output: Name is available

Example 2: Empty string

```
name = ""
if name:
    print("Name is available")
else:
    print("Name is missing")
```

Output: Name is missing

Example 3: Number

```
amount = 0
if amount:
    print("Amount available")
else:
    print("Amount is zero")
```

Output: Amount is zero